C Programming Platform Practice

20 Problems

Programming Practice

Q1–Q2: complete solutions | Q3–Q20: write the implementation

Q1. Positive, Negative or Zero

COMPLETE SOLUTION

Problem Statement

Given an integer N, determine whether it is Positive, Negative, or Zero.

Input Format

A single integer N.

Output Format

Print:

Positive if N > 0

Negative if N < 0

Zero if N == 0

Constraints

-10^9 <= N <= 10^9

Sample Input

-25

Sample Output

Negative

Sample Input

0

Sample Output

Zero

SOLUTION WORKSPACE

```c
#include <stdio.h>

void classifyNumber(long long n) {
    if (n > 0)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");
}

int main(void) {
    long long n;
    scanf("%lld", &n);
    classifyNumber(n);
    return 0;
}
```

Q2. Even or Odd

COMPLETE SOLUTION

Problem Statement

Given an integer N, determine whether it is even or odd.

Input Format

A single integer N.

Output Format

Print Even if N is even; otherwise, print Odd.

Constraints

0 <= N <= 10^9

Sample Input

42

Sample Output

Even

Sample Input

17

Sample Output

Odd

SOLUTION WORKSPACE

```
#include <stdio.h>

int isEven(long long n) {
    return n % 2 == 0;
}

int main(void) {
    long long n;
    scanf("%lld", &n);

    if (isEven(n))
        printf("Even");
    else
        printf("Odd");

    return 0;
}
```

Q3. Electricity Bill Slab

STUDENT SKELETON

Problem Statement

Given the number of electricity units consumed by a household, calculate the bill according to the following rules:

- First 100 units → ₹2 per unit
- Next 100 units → ₹3 per unit
- Next 100 units → ₹5 per unit
- Above 300 units → ₹7 per unit

Input Format

A single integer U, representing units consumed.

Output Format

Print the total electricity bill.

Constraints

0 <= U <= 1000

Sample Input

250

Sample Output

750

SOLUTION WORKSPACE

```
long long calculateElectricityBill(long long units) {
    // TODO: calculate and return the bill
}

int main(void) {
    // TODO: read input
    // TODO: call calculateElectricityBill()
    // TODO: print the result
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q4. Grade Calculator

STUDENT SKELETON

Problem Statement

A student receives marks between 0 and 100. Determine the grade using the following rules:

90 - 100 → A

80 - 89 → B

70 - 79 → C

60 - 69 → D

Below 60 → F

Input Format

A single integer M, representing marks.

Output Format

Print the corresponding grade.

Constraints

0 <= M <= 100

Sample Input

86

Sample Output

B

Sample Input

59

Sample Output

F

SOLUTION WORKSPACE

```
char calculateGrade(int marks) {
    // TODO: determine and return the grade
}


int main(void) {
    // TODO: read marks
    // TODO: call calculateGrade()
    // TODO: print the grade
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q5. Triangle Type Checker

STUDENT SKELETON

Problem Statement

Given three positive integers A, B, and C representing the sides of a triangle, determine whether it is Equilateral, Isosceles, or Scalene. Assume the given sides form a valid triangle.

Input Format

Three space-separated integers A B C.

Output Format

Print the type of triangle.

Constraints

1 <= A, B, C <= 10^6

Sample Input

5 5 8

Sample Output

Isosceles

SOLUTION WORKSPACE

```
void printTriangleType(int a, int b, int c) {
    // TODO: print Equilateral, Isosceles, or Scalene
}

int main(void) {
    // TODO: read a, b, c
    // TODO: call printTriangleType()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q6. Simple Calculator Using Switch

STUDENT SKELETON

Problem Statement

Given two integers A and B and an operator, perform the specified arithmetic operation.
The operator can be:
+
-
*
/
%
For division, assume B is non-zero.

Input Format

The first line contains two integers A and B.
The second line contains a character representing the operator.

Output Format

Print the result of the operation.

Constraints

-10^6 <= A, B <= 10^6
B != 0 for / and %.

Sample Input

20 6
%

Sample Output

2

Sample Input

15 3
/

Sample Output

5

SOLUTION WORKSPACE

```
long long calculateOperation(long long a, long long b, char op) {
    // TODO: use switch and return the result
}


int main(void) {
    // TODO: read a, b, and op
    // TODO: call calculateOperation()
    // TODO: print the result
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q7. Day of the Week

STUDENT SKELETON

Problem Statement

Given a number from 1 to 7, print the corresponding day of the week.
Use the following mapping:
1 → Monday
2 → Tuesday
3 → Wednesday
4 → Thursday
5 → Friday
6 → Saturday
7 → Sunday
Use a switch statement.

Input Format

A single integer D.

Output Format

Print the corresponding day.

Constraints

1 <= D <= 7

Sample Input

5

Sample Output

Friday

SOLUTION WORKSPACE

```
void printDay(int day) {
    // TODO: use switch and print the day
}


int main(void) {
    // TODO: read day
    // TODO: call printDay()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q8. Number of Digits

STUDENT SKELETON

Problem Statement

Given a positive integer N, determine the number of digits in it.

Input Format

A single positive integer N.

Output Format

Print the number of digits in N.

Constraints

1 <= N <= 10^18

Sample Input

58321

Sample Output

5

Sample Input

7

Sample Output

1

SOLUTION WORKSPACE

```
int countDigits(long long n) {
    // TODO: use a loop to count digits
}

int main(void) {
    // TODO: read n
    // TODO: call countDigits()
    // TODO: print the count
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q9. Sum of Digits

STUDENT SKELETON

Problem Statement

Given a positive integer N, calculate the sum of all its digits.

Input Format

A single integer N.

Output Format

Print the sum of the digits of N.

Constraints

1 <= N <= 10^18

Sample Input

58321

Sample Output

19

Sample Input

9005

Sample Output

14

SOLUTION WORKSPACE

```
long long sumDigits(long long n) {
    // TODO: calculate and return digit sum
}

int main(void) {
    // TODO: read n
    // TODO: call sumDigits()
    // TODO: print the sum
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q10. Reverse a Number

STUDENT SKELETON

Problem Statement

Given a positive integer N, reverse its digits and print the resulting number.
Leading zeroes in the reversed number should not be printed.

Input Format

A single positive integer N.

Output Format

Print the reversed number.

Constraints

1 <= N <= 10^9

Sample Input

12345

Sample Output

54321

Sample Input

1200

Sample Output

21

SOLUTION WORKSPACE

```
long long reverseNumber(long long n) {
    // TODO: reverse digits and return result
}


int main(void) {
    // TODO: read n
    // TODO: call reverseNumber()
    // TODO: print the result
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q11. Multiplication Table

STUDENT SKELETON

Problem Statement

Given an integer N, print its multiplication table from 1 to 10.
Each line should follow this format:
N x i = result

Input Format

A single integer N.

Output Format

Print 10 lines containing the multiplication table.

Constraints

1 <= N <= 100

Sample Input

5

Sample Output

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

SOLUTION WORKSPACE

```
void printMultiplicationTable(int n) {
    // TODO: use for loop, 1 through 10
}


int main(void) {
    // TODO: read n
    // TODO: call printMultiplicationTable()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q12. Sum of First N Natural Numbers

STUDENT SKELETON

Problem Statement

Given a positive integer N, calculate the sum:

1 + 2 + 3 + ... + N

You must use a loop to calculate the sum.

Input Format

A single integer N.

Output Format

Print the sum.

Constraints

1 <= N <= 10^6

Sample Input

10

Sample Output

55

SOLUTION WORKSPACE

```
long long sumFirstN(long long n) {
    // TODO: calculate 1 + 2 + ... + n
}

int main(void) {
    // TODO: read n
    // TODO: call sumFirstN()
    // TODO: print the sum
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q13. Count Even and Odd Numbers

STUDENT SKELETON

Problem Statement

Given N integers one at a time, count how many of them are even and how many are odd.

You must process the numbers using a loop. Do not use arrays.

Input Format

The first line contains an integer N.

The second line contains N space-separated integers.

Output Format

Print two lines:

Even: X

Odd: Y

Constraints

1 <= N <= 10^5

-10^9 <= X <= 10^9

Sample Input

6

10 7 8 3 12 15

Sample Output

Even: 3

Odd: 3

Note: This problem requires processing values directly from input; storing them in an array is not necessary.

SOLUTION WORKSPACE

```
void countEvenOdd(int n) {
    int even = 0, odd = 0;

    // TODO: read n integers
    // TODO: update even / odd counts

    printf("Even: %d\n", even);
    printf("Odd: %d", odd);
}

int main(void) {
    // TODO: read n
    // TODO: call countEvenOdd()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q14. Print Numbers Except Multiples of 3

STUDENT SKELETON

Problem Statement

Given an integer N, print all numbers from 1 to N except those that are divisible by 3.
You must use the continue statement.

Input Format

A single integer N.

Output Format

Print the valid numbers separated by spaces.

Constraints

1 <= N <= 10^5

Sample Input

10

Sample Output

1 2 4 5 7 8 10

SOLUTION WORKSPACE

```
void printExceptMultiplesOf3(int n) {
    // TODO: use continue to skip multiples of 3
}

int main(void) {
    // TODO: read n
    // TODO: call printExceptMultiplesOf3()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q15. Find the First Number Divisible by 7

STUDENT SKELETON

Problem Statement

Given an integer N, find the first number greater than or equal to 1 and less than or equal to N that is divisible by 7.

If no such number exists, print -1.

You must use the break statement.

Input Format

A single integer N.

Output Format

Print the first number divisible by 7, or -1 if none exists.

Constraints

1 <= N <= 10^6

Sample Input

30

Sample Output

7

Sample Input

5

Sample Output

-1

SOLUTION WORKSPACE

```
int firstDivisibleBy7(int n) {
    // TODO: search from 1 to n
    // TODO: use break after the first match
    // TODO: return -1 if no match
}

int main(void) {
    // TODO: read n
    // TODO: call firstDivisibleBy7()
    // TODO: print the result
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q16. Password Attempts

STUDENT SKELETON

Problem Statement

A system allows a user a maximum of 3 attempts to enter a numeric password.
The correct password is 1234.
Read the password attempts one by one.
If the correct password is entered, print Access Granted and stop immediately.
If all three attempts are incorrect, print Access Denied.
You must use a loop and break.

Input Format

Three integers, each representing a password attempt.

Output Format

Print Access Granted if the correct password is entered within three attempts.
Otherwise, print Access Denied.

Constraints

0 <= password <= 9999

Sample Input

1111
2222
1234

Sample Output

Access Granted

Sample Input

1111
2222
3333

Sample Output

Access Denied

SOLUTION WORKSPACE

```
void checkPasswordAttempts(void) {
    // TODO: read three attempts
    // TODO: use break when password is correct
    // TODO: print the required result
}

int main(void) {
    // TODO: call checkPasswordAttempts()
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q17. Keep Reading Until Zero

STUDENT SKELETON

Problem Statement

Read integers continuously until the user enters 0.
Calculate and print the sum of all non-zero numbers entered.
You must use a while loop.

Input Format

A sequence of integers ending with 0.

Output Format

Print the sum of all numbers before 0.

Constraints

The input contains at least one 0.
Each non-zero value satisfies:
-10^6 <= N <= 10^6

Sample Input

10
20
-5
15
0

Sample Output

40

SOLUTION WORKSPACE

```
long long sumUntilZero(void) {
    // TODO: use while to read until 0
    // TODO: return sum of non-zero values
}

int main(void) {
    // TODO: call sumUntilZero()
    // TODO: print the result
    return 0;
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q18. Menu-Driven Program

STUDENT SKELETON

Problem Statement

Create a menu-driven program using a switch statement.
The menu is:
1. Square
2. Cube
3. Check Even/Odd
4. Exit
Read the user's choice.
For choices 1, 2, and 3, read an integer N and perform the corresponding operation.
For choice 4, terminate the program.

Input Format

The first line contains an integer choice.
If choice is 1, 2, or 3, the next line contains an integer N.

Output Format

For choice:
1: Print the square of N
2: Print the cube of N
3: Print Even or Odd
4: Print Program Ended
For any other choice, print Invalid Choice.

Constraints

-1000 <= N <= 1000
1 <= choice <= 5

Sample Input

2
5

Sample Output

125

Sample Input

3
18

Sample Output

Even

SOLUTION WORKSPACE

```
void runMenu(int choice, int n) {
    // TODO: use switch for the menu choices
    // TODO: implement each required operation
}

int main(void) {
    // TODO: read choice
    // TODO: read n when required
    // TODO: call runMenu()
    return 0;
}
```

Students complete the TODO comments. main() is intentionally left for you.

Q19. Factorial Using Do-While

STUDENT SKELETON

Problem Statement

Given a non-negative integer N, calculate N! using a do-while loop.

Recall:

N! = 1 × 2 × 3 × ... × N

Also, 0! = 1.

Input Format

A single integer N.

Output Format

Print N!.

Constraints

0 <= N <= 12

Sample Input

5

Sample Output

120

Sample Input

0

Sample Output

1

SOLUTION WORKSPACE

```
long long factorialUsingDoWhile(int n) {
    // TODO: calculate n! using do-while
}
```

Student: complete the TODO comments. main() is intentionally left for you.

Q20. Special Number Checker

Problem Statement

Given a positive integer N, determine whether it is a Special Number.
A number is considered special if:
1. It is a two-digit number.
2. The sum of its digits is equal to 10.
Print Special if both conditions are satisfied; otherwise, print Not Special.

Input Format

A single integer N.

Output Format

Print:
Special if N satisfies the conditions.
Not Special otherwise.

Constraints

1 <= N <= 9999

Sample Input

64

Sample Output

Special

Sample Input

123

Sample Output

Not Special

Sample Input

91

Sample Output

Special

SOLUTION WORKSPACE

STUDENT SKELETON

```
int isSpecialNumber(int n) {
    // TODO: check both conditions
    // TODO: return 1 if Special, otherwise 0
}
```

Student: complete the TODO comments. main() is intentionally left for you.